EXPERIMENT – 5

Title: Broadcast Socket Programming

Objective:

To implement broadcast socket programming for sending a message to all computers in a network.

Software / Tools Required

- Operating System: Linux
- Programming Language: Python 3
- Custom Module: netpack

Theory

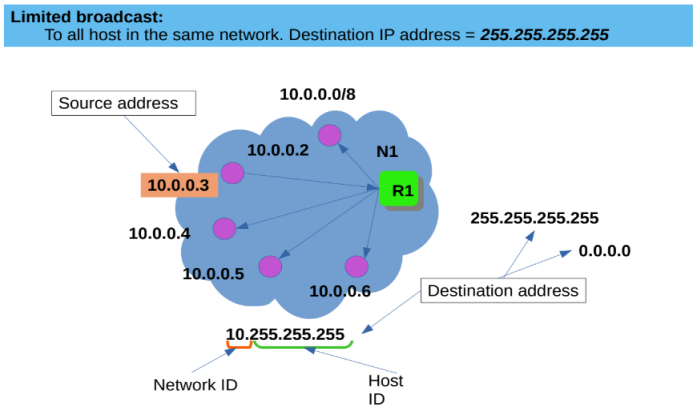
Broadcast communication in computer networks means sending a message from one device to all devices present in the same network at the same time. This is done using a special **broadcast IP address** so that every host connected to the network can receive the message.

One common broadcast address is **255.255.255.255**, which is called **limited broadcast**. It sends the message to all hosts within the same local network and routers normally do not forward this message to other networks.

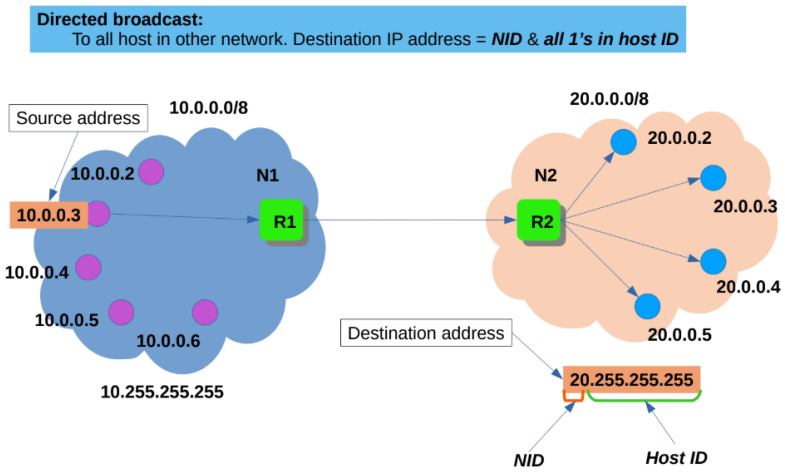
Another type is **directed broadcast**, where the address contains the **network ID and all 1's in the host part**. For example, for the network **20.0.0.0/8**, the broadcast address becomes **20.255.255.255**, which sends the message to all hosts in that particular network.

In this experiment, a **UDP broadcast socket** is used where the server sends broadcast messages and multiple clients in the network receive the same message simultaneously.

Limited Broadcast



Directed Broadcast



Procedure / Algorithm

Server Side

1. Import required modules.
2. Create UDP broadcast server socket.
3. Define broadcast IP address **255.255.255.255** and client port.
4. Generate random number to send as broadcast message.
5. Send the message to broadcast address.
6. Wait for a few seconds and repeat broadcasting.
7. If user interrupts the program, send **"close"** message and terminate connection.

Client Side

1. Import required modules.
2. Create UDP broadcast client socket.
3. Bind the socket to the specified port.
4. Wait for broadcast message from server.
5. Receive message from server and display it.
6. If the received message is **"close"**, close the connection and terminate the program.

Program Code

Server Program:

```python
import netpack as npk
import random
import time

broadcast_ip_1='255.255.255.255'
client_port=10999
addr=(broadcast_ip_1, client_port)

# Step-1
server_socket=npk.create_udp_broadcast_server_socket()

while True:
    try:
        data=random.randint(0,100)
        npk.send_data_to_udp_client(server_socket, data, addr)
        print("[SENT] Broadcast Message")
        time.sleep(2)
    except KeyboardInterrupt:
        print('Program Interrupted')
        npk.send_data_to_udp_client(server_socket, 'close', addr)
        npk.close_socket_connection(server_socket)
        break
```

Client Program:

```python
import netpack as npk
import random
import time

client_port=10999

# Step-1
client_socket=npk.create_udp_broadcast_client_timeoutsocket(client_port)


T1=time.time()
while True:
    try:
        (data, ADDR)=npk.read_data_from_udp_server(client_socket)
        print('[=] Message from server : {}'.format(data))
    except KeyboardInterrupt:
        print('Program Interrupted')
        npk.close_socket_connection(client_socket)
        break
    except:
        pass
    else:
        if data=='close':
            print('Server Closed')
            npk.close_socket_connection(client_socket)
            break
```

Output:

Server Side:

```
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-5$ python3 5_UDP_Broadcast_server.py
[SENT] Broadcast Message
[SENT] Broadcast Message
[SENT] Broadcast Message
[SENT] Broadcast Message
[SENT] Broadcast Message
[SENT] Broadcast Message
[SENT] Broadcast Message
[SENT] Broadcast Message
[SENT] Broadcast Message
[SENT] Broadcast Message
[SENT] Broadcast Message
[SENT] Broadcast Message
```

Client Side:

```
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-5$ python3 5_UDP_Broadcast_client_1.py
[=] Message from server : 96
[=] Message from server : 72
[=] Message from server : 48
[=] Message from server : 99
[=] Message from server : 12
[=] Message from server : 1
[=] Message from server : 35
[=] Message from server : 48
[=] Message from server : 11
[=] Message from server : 53
[=] Message from server : 17
[=] Message from server : 59
```

Result

The broadcast socket program was executed successfully and the server message was received by multiple clients in the network.

Conclusion

From this experiment we learned how **broadcast communication works using UDP sockets** and how a single server can send the same message to multiple clients in a network.